

We beat a neural prompt compressor with 1970s NLP

A chat about a thing we tried at the gateway, and whether the numbers hold up.

Mesh API routing team. June 2026. Real measurements, on a laptop.

Riya asks

what were you heads down on all week

Sam

prompt compression, but for the gateway. you know how Mesh sits in front of like 300 models and every request goes through it first.

Sam

if you can cut the dead weight out of a prompt before it hits the model, you save tokens and you dodge the lost in the middle thing where models ignore stuff buried in the center.

Riya asks

isn't that solved though. just use LLMingua or one of those

Sam

that was my assumption too. but all of those run a small language model over your prompt to score which tokens matter. so the compressor itself needs a forward pass.

Sam

fine inside one app. at the gateway it is rough, because you pay that cost on every single request, for every tenant. it adds straight to the time before the first token comes back, and it wants its own GPU.

Riya asks

how much does that actually cost you

Sam

here is the part that sold me. in the LongLLMingua paper, one of these methods (Selective Context) had a net speedup of 0.6x. so after you count

the compressor, the whole request got slower. you added a model to save time and it cost you time.

Riya asks

ok that is funny. so what did you do instead

Sam

went backwards on purpose. classical stuff. BM25, TF-IDF, TextRank, centroid plus MMR. old information retrieval methods. no model, they just rank sentences. run in well under a millisecond on a CPU.

Sam

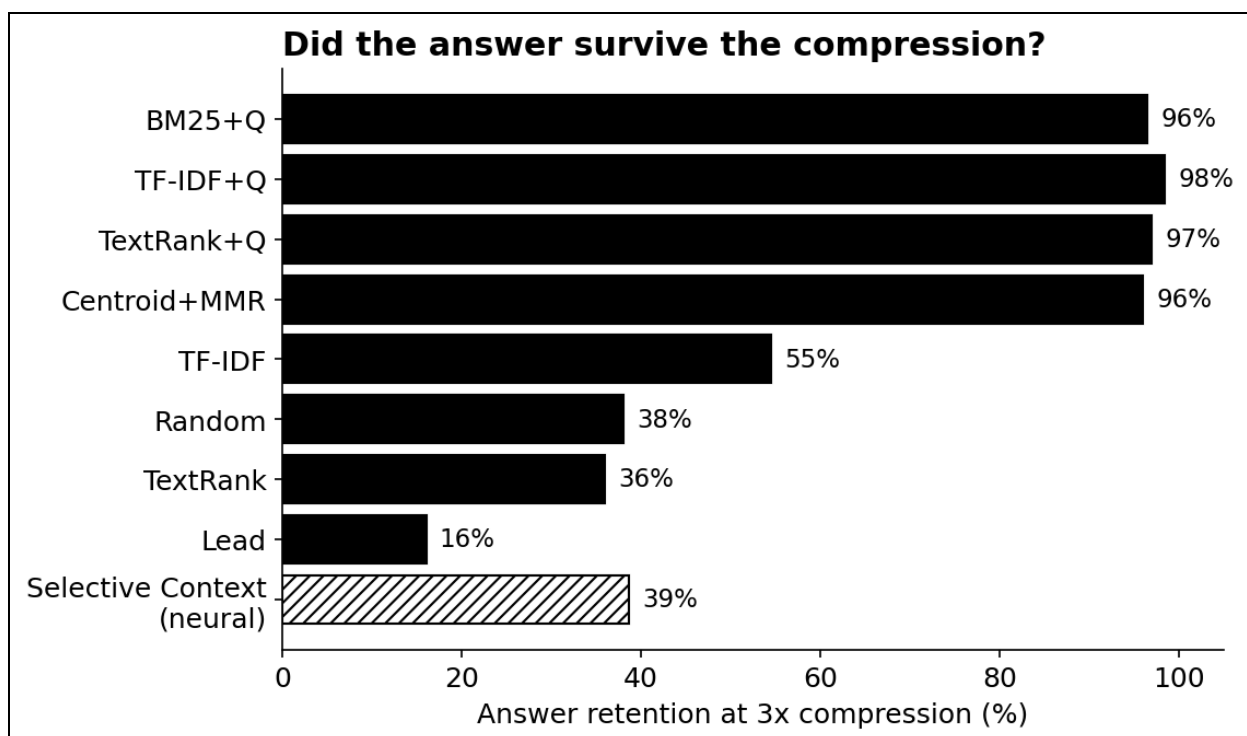
built a benchmark from SQuAD where the answer is hidden in the middle of a pile of distractor paragraphs, then checked: after compressing, did the actual answer survive. ran every method head to head against a real neural baseline on the same laptop.

Riya asks

and

Sam

this is the chart. keep the answer or lose it, at 3x compression.



Answer retention at 3x compression. The hatched bar is the neural method.

Riya asks

wait the old methods are at the top and the neural one is near the bottom

Sam

yeah. and the thing that actually splits the chart is not classical vs neural. it is whether the method looks at the question.

Sam

the query aware classical ones all sit around 96 to 98 percent. the neural baseline gets 38.7 percent, basically tied with picking sentences at random.

Sam

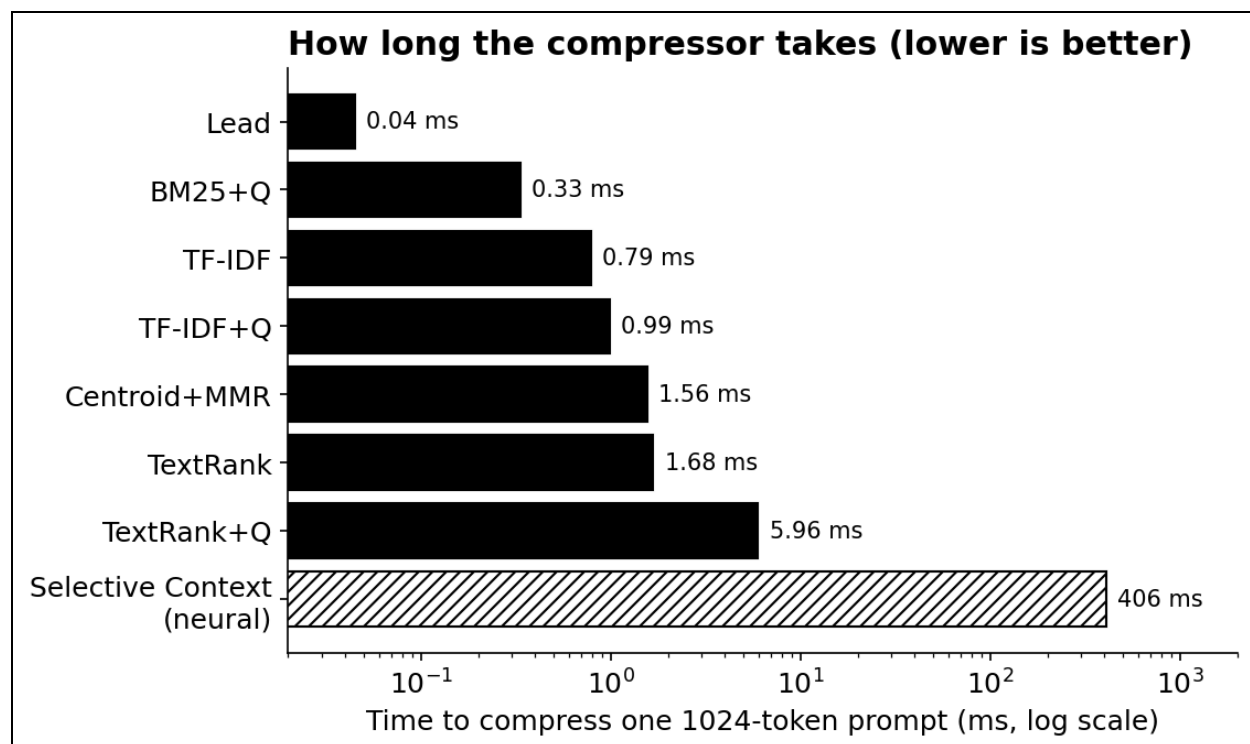
because Selective Context never reads the question. it just keeps whatever tokens look surprising. but a gateway request almost always comes with a question. once you use it, even plain TF-IDF jumps from 54.5 to 98.5 percent.

Riya asks

alright but accuracy is half of it. what about speed and cost

Sam

that part is not close at all.



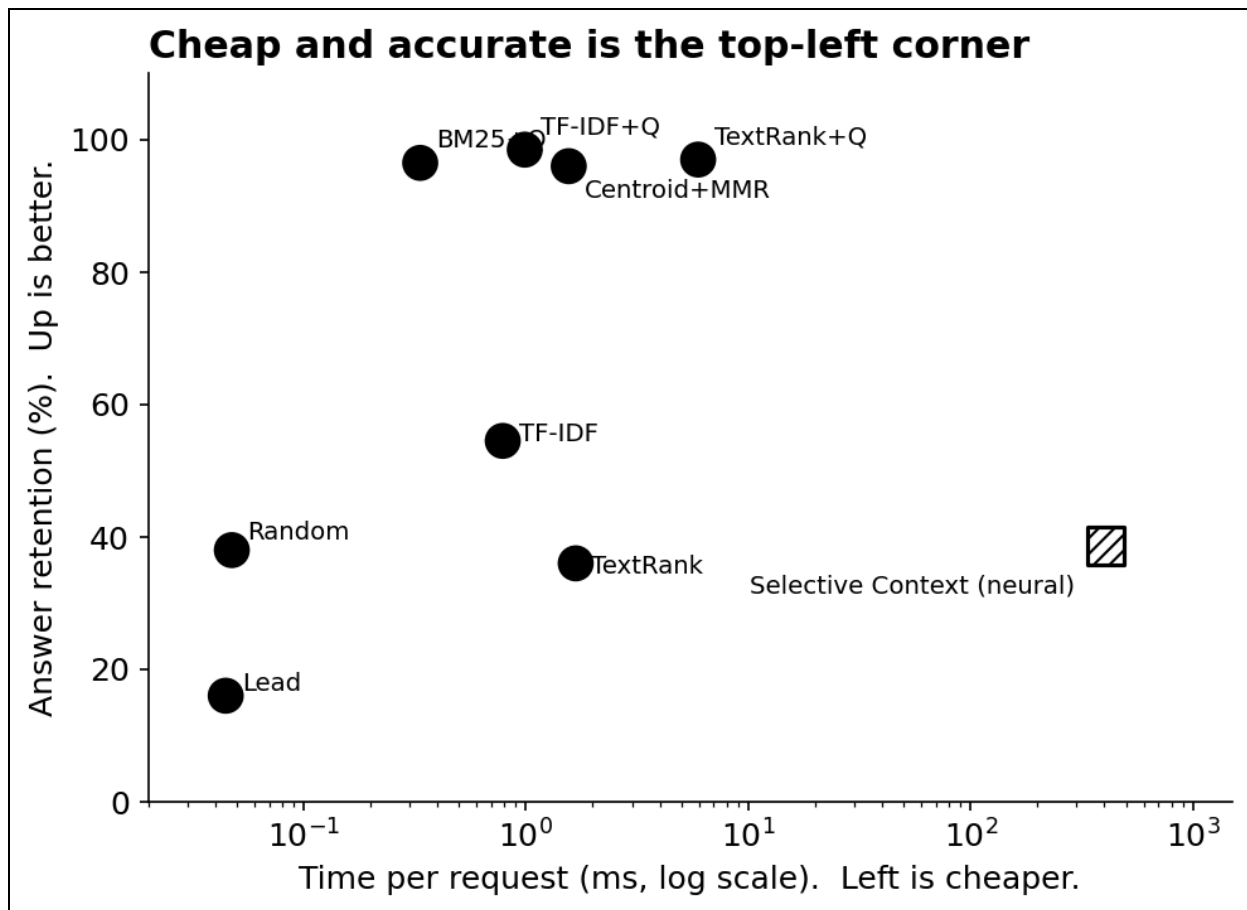
Time to compress one 1024-token prompt. Note the log scale.

Sam

classical lands around 0.33 ms. the neural one takes 406 ms on the same machine. about 1213 times slower, and the gap gets worse on longer prompts because it reads every token. plus it carries 312 MB of weights. the classical ones carry none.

Sam

if you put both on one plot, cheap and accurate is the top left corner, and the old methods own it.



Top left is cheap and accurate. The neural method is the lonely square.

Riya asks

ok real question. did you actually run all this or is this a nice theory with made up numbers

Sam

fair, and I would ask the same. it is real. I ran the full thing, 200 questions, and the neural part is a real distilgpt2 doing real forward passes, it took about 6 minutes on the laptop.

Sam

then to be sure it was not just a lucky random draw, I ran it again on a fresh sample with a different seed. BM25 went 96.5 to 95. TF-IDF+Q 98.5 to 96.7. the neural one 38.7 to 40. same story, different data. nothing cherry picked.

Riya asks

good. now tell me where it is weak, because that chart looks too clean

Sam

three honest things.

Sam

one, my neural baseline is a small model on CPU. that is a fair same machine test, but on a GPU its latency would drop a lot, so the 1213x is specific to this setup. the accuracy gap is the real point anyway, and that is about it ignoring the question, not its size.

Sam

two, I measured whether the answer text survives the cut, not whether a model then answers correctly. strong proxy, not the final word. I wrote the end to end version that runs answers through Mesh, but I have not run it yet, no key handy.

Sam

three, these methods only keep or drop whole sentences. they cannot rewrite. and it is English SQuAD with short answers. multi hop or other languages might move the numbers.

Riya asks

that is the part I trust the most honestly. so what is the one line

Sam

people study prompt compression like a modeling problem. at the gateway it is a systems problem, and the cheapest tool wins. a 1970s ranking function beat a neural net here, mostly because it bothered to read the question.

Riya asks

put that in the post

Sam

already did.

The paper, the benchmark, and all the code are open. It reproduces offline. The only network call is a one time download of SQuAD and a small model.

Paper (PDF): [\[link\]](#) · Code and benchmark: [\[link\]](#) · [meshapi.ai](#)